

Module 2

Q. Activation Functions: Tanh, Logistic, Linear, Softmax, ReLU, Leaky ReLU

=>

Introduction

1. Activation functions are mathematical functions used in neural networks to produce output from a neuron.
2. Activation functions help the network learn complex patterns by introducing non-linearity.
3. Activation functions decide whether a neuron should be activated or not.
4. Different activation functions are used for different types of problems.

1. Tanh (Hyperbolic Tangent) Function

1. The Tanh function is a non-linear activation function.
2. The output range of the Tanh function is between -1 and +1.
3. The mathematical formula is
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$
4. The Tanh function is zero-centered, which helps in faster learning.
5. The function is commonly used in hidden layers of neural networks.
6. Example: Tanh is used in recurrent neural networks for sequence prediction tasks.

2. Logistic (Sigmoid) Function

1. The Logistic function is also called the Sigmoid function.
2. The output range of the Sigmoid function is between 0 and 1.
3. The mathematical formula is
$$\sigma(x) = \frac{1}{1+e^{-x}}.$$
4. The function is mainly used for binary classification problems.
5. The output can be interpreted as probability.
6. Example: Sigmoid is used in email spam detection to predict spam probability.

3. Linear Activation Function

1. The Linear activation function produces output equal to input.
2. The mathematical formula is ($f(x) = x$).
3. The output range is from negative infinity to positive infinity.
4. The function does not introduce non-linearity in the model.
5. Linear activation is mainly used in regression problems.
6. Example: Predicting house price using neural networks uses linear activation in the output layer.

4. Softmax Function

1. The Softmax function is used for multi-class classification problems.
2. The Softmax function converts values into probability distribution.
3. The output values always sum to 1.
4. The mathematical formula is

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}.$$

5. The function is usually used in the output layer of neural networks.
6. Example: Image classification into cat, dog, or bird uses Softmax.

5. ReLU (Rectified Linear Unit)

1. ReLU is one of the most commonly used activation functions in deep learning.
2. The mathematical formula is $f(x) = \max(0, x)$.
3. The output becomes zero for negative inputs.
4. The output becomes equal to input for positive inputs.
5. ReLU helps to reduce the vanishing gradient problem.
6. ReLU is widely used in convolutional neural networks (CNNs).
7. Example: ReLU is used in image recognition systems.

6. Leaky ReLU

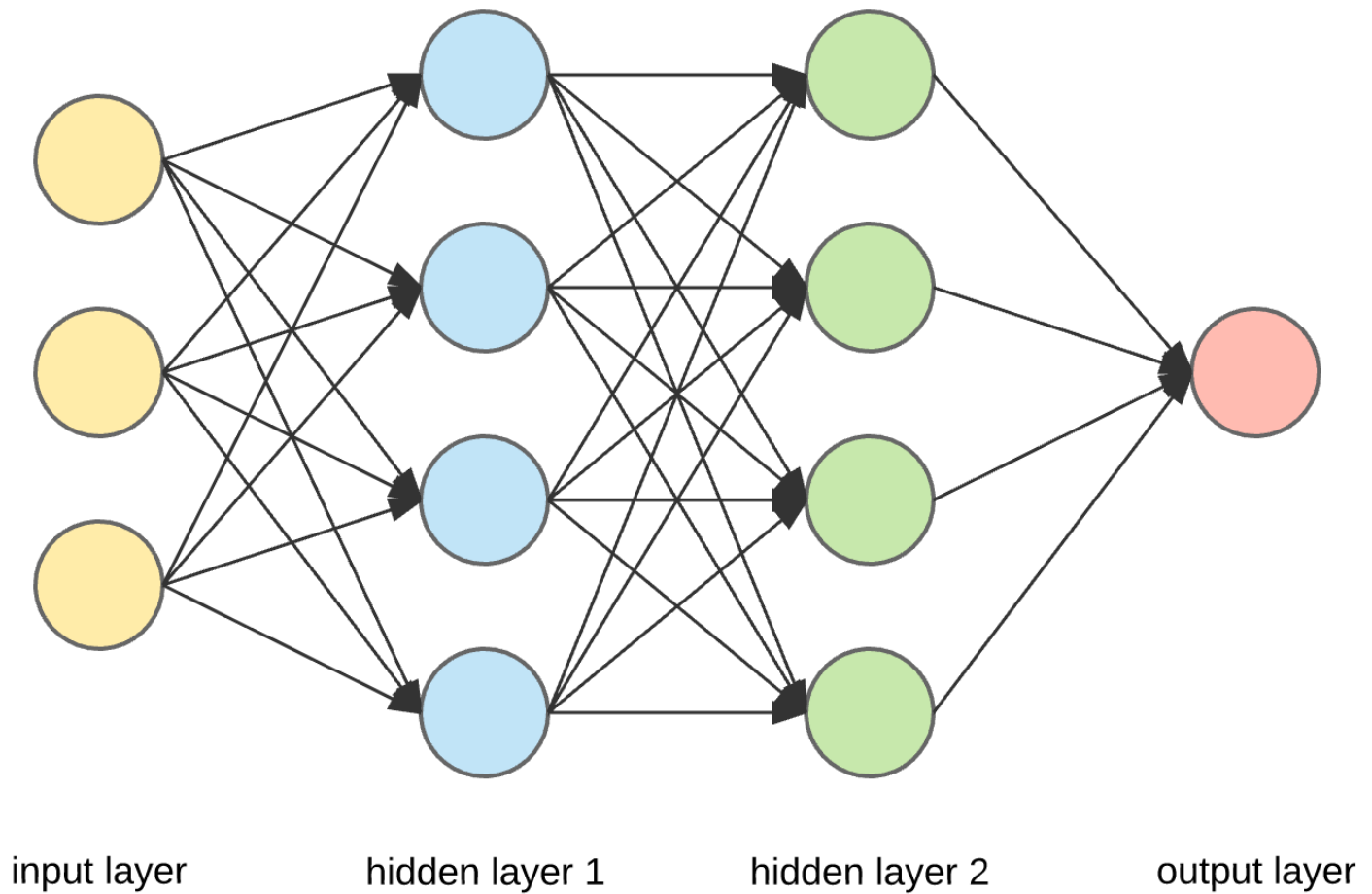
1. Leaky ReLU is an improved version of ReLU.
2. Leaky ReLU allows a small value for negative inputs instead of zero.
3. The mathematical formula is

$$f(x) = x \text{ if } x > 0, \text{ otherwise } f(x) = \alpha x.$$

4. The symbol (alpha) is a small constant like 0.01.
5. Leaky ReLU solves the dying ReLU problem.
6. Example: Leaky ReLU is used in deep neural networks where many neurons become inactive with ReLU.

Q. Multi-Layered Feed Forward Neural Network (MLFFNN)

=>



Introduction

1. A Multi-Layered Feed Forward Neural Network is a type of artificial neural network that contains more than one layer of neurons.
2. The network is called feed forward because information moves only in one direction from input to output.
3. The network does not contain any feedback or loops between neurons.
4. The MLFFNN is one of the most commonly used architectures in deep learning.
5. The model is capable of solving complex non-linear problems.

Architecture of MLFFNN

1. Input Layer

1. The input layer receives data from the external environment.
2. Each neuron in the input layer represents one feature of the dataset.
3. The input layer does not perform any computation.

4. Example: In image recognition, pixel values act as inputs.

2. Hidden Layers

1. Hidden layers are placed between the input layer and the output layer.
2. A network can have one or more hidden layers.
3. Each neuron in a hidden layer performs weighted summation and activation.
4. Hidden layers help the network learn complex patterns.
5. Deep neural networks contain many hidden layers.
6. Example: Speech recognition systems use multiple hidden layers to learn sound patterns.

3. Output Layer

1. The output layer produces the final result of the network.
2. The number of neurons depends on the type of problem.
3. A single neuron is used for binary classification.
4. Multiple neurons are used for multi-class classification.
5. Example: Digit recognition from 0–9 uses 10 output neurons.

Working of MLFFNN

1. The network receives input data at the input layer.
2. Each neuron multiplies input values with weights.
3. The weighted sum is passed through an activation function.
4. The output of one layer becomes the input to the next layer.
5. The process continues until the output layer produces the final result.
6. The network learns by adjusting weights using training algorithms like backpropagation.

Mathematical Representation

1. Each neuron computes ($z = \text{sum of } w_i x_i + b$).
2. The activation function produces output ($y = f(z)$).
3. The same process is repeated across multiple layers.

Characteristics

1. The MLFFNN can learn both linear and non-linear relationships.
2. The network supports supervised learning.
3. The network requires large datasets for training.
4. The performance improves with more hidden layers and neurons.
5. The network uses activation functions such as ReLU, Sigmoid, or Tanh.

Advantages

1. The network can solve complex real-world problems.
2. The network can approximate any function with enough neurons.
3. The model is flexible and widely used.

Limitations

1. The training process requires high computational power.
2. The network may suffer from overfitting if data is limited.
3. The training time increases with network depth.

Applications

1. The MLFFNN is used in image recognition systems.
2. The MLFFNN is used in speech recognition systems.
3. The MLFFNN is used in medical diagnosis prediction.
4. The MLFFNN is used in financial forecasting.

Q. Gradient Descent (GD) and Its Types

=>

Introduction

1. Gradient Descent is a method used to reduce error in machine learning and neural networks.
2. Gradient Descent helps the model find the best weight values.
3. The algorithm updates weights step by step to reach minimum error.
4. The direction of movement is always towards decreasing loss.
5. The learning rate decides how big the step size is.

Basic Formula

1. The weight update formula is ($w = w - \eta \times \text{gradient}$).
2. The symbol (η) represents the learning rate.
3. The gradient shows how much error changes with respect to weight.

Types of Gradient Descent

1. Batch Gradient Descent

1. Batch Gradient Descent uses the entire dataset to update weights.
2. The method is slow when the dataset is very large.
3. The method is stable because it uses complete information.
4. Example: Small datasets training uses batch GD.

2. Stochastic Gradient Descent (SGD)

1. Stochastic Gradient Descent updates weights using only one sample at a time.
2. The method is faster than batch gradient descent.
3. The updates are noisy because they depend on single samples.
4. The noise sometimes helps find better solutions.
5. Example: Real-time learning systems use SGD.

3. Mini-Batch Gradient Descent

1. Mini-batch Gradient Descent uses a small group of samples for each update.
2. The method is faster than batch GD and more stable than SGD.
3. Mini-batch is the most commonly used method in deep learning.
4. Example: Batch size 32 or 64 training uses mini-batch GD.

Momentum Based Methods

4. Momentum Gradient Descent

1. Momentum adds the previous update information to the current update.
2. Momentum helps the model move faster in the correct direction.
3. Momentum reduces oscillations during training.
4. Example: Deep neural network training uses momentum.

5. Nesterov Accelerated Gradient (NAG)

1. Nesterov is an improved version of momentum.
2. Nesterov looks ahead before calculating the gradient.
3. Nesterov gives faster convergence than normal momentum.
4. Example: Complex deep learning models use NAG.

Adaptive Learning Rate Methods

6. AdaGrad

1. AdaGrad changes the learning rate automatically for each parameter.
2. Frequently updated weights get smaller learning rates.
3. Rarely updated weights get larger learning rates.
4. AdaGrad works well for sparse data problems.
5. Example: Text processing models use AdaGrad.

7. RMSProp

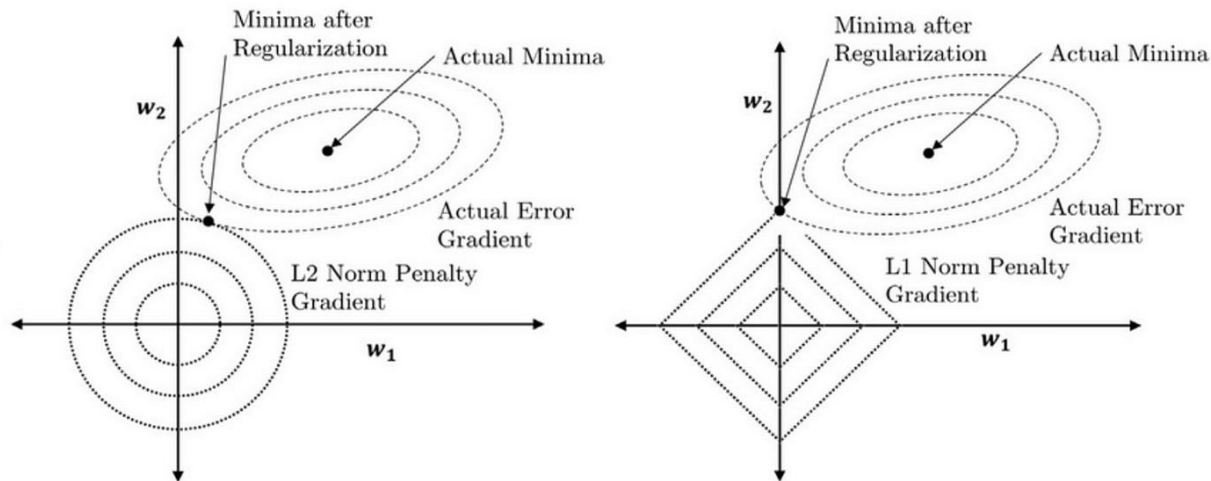
1. RMSProp improves AdaGrad by preventing learning rate from becoming too small.
2. RMSProp uses moving average of gradients.
3. RMSProp works well for changing data conditions.
4. Example: Recurrent neural networks use RMSProp.

8. Adam (Adaptive Moment Estimation)

1. Adam combines momentum and RMSProp methods.
2. Adam uses both average gradient and squared gradient.
3. Adam gives fast and stable training.
4. Adam is the most widely used optimizer today.
5. Example: Image classification and NLP models use Adam.

Q. Regularization Methods: L1 and L2 Regularization

=>



Introduction to Regularization

1. Regularization is a technique used to reduce overfitting in machine learning and deep learning models.
2. Overfitting happens when a model learns training data too well but performs poorly on new data.
3. Regularization adds a penalty term to the loss function to control model complexity.
4. Regularization helps in improving the generalization ability of the model.
5. L1 and L2 are the most commonly used regularization methods.

L1 Regularization (Lasso Regularization)

Basic Idea

1. L1 regularization adds the absolute values of weights as a penalty to the loss function.
2. The method tries to reduce some weights to exactly zero.
3. The method performs feature selection automatically.

Mathematical Formula

1. The L1 penalty term is $\lambda \sum |w|$.
2. The symbol λ represents the regularization parameter.
3. The total loss becomes Loss + L1 penalty.

Characteristics

1. L1 regularization produces sparse models.

2. Sparse models contain many zero weights.
3. The method helps remove irrelevant features.
4. The method is useful when the dataset has many features.

Example

1. L1 regularization is used in text classification where many words are not important.
2. L1 helps select only useful words for prediction.

L2 Regularization (Ridge Regularization)

Basic Idea

1. L2 regularization adds the square of weights as a penalty to the loss function.
2. The method reduces weights but does not make them exactly zero.
3. The method distributes weight values smoothly.

Mathematical Formula

1. The L2 penalty term is $\lambda \sum w^2$.
2. The symbol λ represents the regularization parameter.
3. The total loss becomes Loss + L2 penalty.

Characteristics

1. L2 regularization prevents weights from becoming very large.
2. The method improves model stability.
3. The method works well when all features are important.
4. The method reduces overfitting without removing features completely.

Example

L2 regularization is used in image recognition models where all pixels contribute information.

Difference Between L1 and L2

1. L1 uses absolute values of weights while L2 uses squared weights.
2. L1 produces sparse models while L2 produces small but non-zero weights.
3. L1 performs feature selection while L2 does not remove features.
4. L1 is useful when many features are irrelevant while L2 is useful when most features are useful.